	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.C. René Adrián Dávila Pérez

Asignatura: Programación orientada a objetos

Grupo: 07

No de práctica(s): 09

Integrante(s): 424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada: 1

Semestre: 2027 - 1

Fecha de entrega: 30 de octubre de 2026

Observaciones:

Calificación:

Índice

1. Introducción **2**

1.1. Planteamiento del Problema 2

1.2. Motivación 2

1.3. Objetivos 2

2. Marco Teórico **2**

3. Desarrollo **3**

3.1. Descripción de la Implementación 3

3.2. Modelado Estructural: Diagrama de Clases y Cardinalidades 3

3.3. Modelado Dinámico: Diagramas de Secuencia del Sistema 4

3.4. Metodología de Modelado y Verificación 4

4. Resultados **4**

4.1. Diagramas Estructurales de Clases 5

4.2. Diagramas Dinámicos de Secuencia 6

5. Conclusiones **9**

6. Referencias **10**

1. Introducción

1.1 Planteamiento del Problema

Durante el diseño y desarrollo de sistemas de software orientados a objetos, la falta de una representación gráfica estandarizada de la arquitectura estática y dinámica genera ambigüedades en la implementación, acoplamiento acentuado entre componentes y dificultades para estructurar lógicamente las relaciones de cardinalidad y el flujo temporal de mensajes. El problema radica en la necesidad de traducir los requerimientos funcionales de diversos escenarios de programación, desde clases independientes con atributos encapsulados hasta jerarquías complejas con clases abstractas, interfaces y colecciones dinámicas, en modelos visuales formales que aseguren la consistencia arquitectónica antes de escribir código fuente.

1.2 Motivación

La integración formal del Lenguaje de Modelado Unificado (UML) resulta indispensable para visualizar, especificar, construir y documentar con rigor científico los artefactos de un sistema de software. Plasmar gráficamente la estructura de clases, los atributos, las operaciones, los niveles de visibilidad y las multiplicidades permite identificar de antemano violaciones a los principios de diseño modular y encapsulamiento. Asimismo, modelar la secuencia temporal de las llamadas e interacciones entre objetos clarifica el paso de mensajes, el despacho dinámico y el control de flujos alternativos, garantizando que el diseño algorítmico cumpla estrictamente con las reglas de negocio establecidas.

1.3 Objetivos

- **Objetivo General:** Utilizar UML como herramienta para diseñar soluciones de software para un lenguaje de programación orientado a objetos.
- **Objetivos Específicos:**

    1. Diseñar el diagrama estático de clases del sistema modelando las jerarquías de herencia, las interfaces, los atributos, los métodos y las cardinalidades de asociación entre entidades.
    2. Construir los diagramas dinámicos de secuencia para representar el intercambio cronológico de mensajes, llamadas a métodos y marcos de control condicional entre los objetos principales durante la ejecución de los casos de uso.
    3. Aplicar las normas de notación estándar de UML 2.5 para estandarizar la documentación técnica de la arquitectura orientada a objetos desarrollada en el proyecto.

2. Marco Teórico

El Lenguaje de Modelado Unificado (UML) constituye el estándar universal de la industria para la especificación, visualización, construcción y documentación de los artefactos que componen un sistema de software orientado a objetos [1]. UML clasifica los diagramas en dos vertientes fundamentales: estructurales y de comportamiento [1, 2].

Los diagramas estructurales modelan la vista estática del sistema, enfatizando los elementos que componen

la arquitectura independientemente del tiempo [1]. El Diagrama de Clases es el pilar central del diseño orientado a objetos; describe las clases del sistema, sus atributos, operaciones y las relaciones estáticas entre ellas, tales como la asociación, la generalización (herencia), la realización (interfaces) y la dependencia [2, 3]. Un aspecto fundamental en las asociaciones de clases es la cardinalidad o multiplicidad, la cual define el número de instancias de una clase que pueden relacionarse con una instancia de otra clase (por ejemplo, expresiones como 0..1, 1..* o 0..10), delimitando los topes estructurales de almacenamiento en arreglos y estructuras dinámicas [2, 3].

Por otro lado, los diagramas de comportamiento modelan la dinámica del sistema y los cambios de estado a lo largo del tiempo [1]. El Diagrama de Secuencia detalla la interacción entre los objetos organizada cronológicamente en una dimensión temporal [2]. En este diagrama, los objetos se representan como líneas de vida verticales y las interacciones se despliegan mediante mensajes síncronos y asíncronos que activan barras de control de ejecución, permitiendo visualizar con precisión el flujo de llamadas, la evaluación de condiciones lógicas mediante marcos de control y el retorno de valores entre los componentes del software [2, 3].

3. Desarrollo

3.1 Descripción de la Implementación

1. **Módulos de Prácticas Previas (P5 a P8):**
   - **Módulo Fecha y Empleado:** Modelado de entidades concretas para el control de atributos temporales y salariales con validación de estado.
   - **Módulo Geometría (Cuadrilátero y Punto):** Representación de agregaciones entre coordenadas espaciales y polígonos especializados (Trapezoide, Paralelogramo, Rectángulo y Cuadrado).
   - **Módulo Huella de Carbono:** Implementación de contratos de interfaz (`«interface» HuellaCarbono`) sobre entidades heterogéneas (Edificio, Auto, Bicicleta).
2. **Simulador de Sistema Hospitalario (Etapa 2):**
   - **Paquete hospital.interfaces:** Define la interfaz AtencionMedica empaquetada externamente en la biblioteca `lib/AtencionMedica.jar`.
   - **Paquete hospital.pacientes:** Integra la clase abstracta Paciente y las subclases concretas PacienteGeneral y PacienteEspecialidad.
   - **Paquete hospital.personal:** Contiene las superclases abstractas Medico y Enfermero, especializándose en MedicoGeneral, MedicoEspecialista, EnfermeroEspecialista y JefeEnfermeria.
   - **Paquete hospital.sistema:** Aloja el controlador central Sistema y la interfaz interactiva Main.

3.2 Modelado Estructural: Diagrama de Clases y Cardinalidades

Para dar solución a los requerimientos de modelado, estructuramos el diseño estático mediante diagramas de clases que reflejan la evolución arquitectónica del software. En las implementaciones modulares de menor escala (como los módulos de fechas, empleados y figuras geométricas), definimos clases con atributos encapsulados y métodos de acceso, estableciendo asociaciones de agregación y composición con clases principales que gestionan la ejecución mediante arreglos o referencias directas.

En los sistemas de mayor complejidad (como el simulador de entorno hospitalario), el diagrama estático modela superclases abstractas (Paciente, Medico, Enfermero) que centralizan atributos comunes y algoritmos invariantes, vinculadas mediante herencia con subclases especializadas y conectadas a través de una interfaz de atención médica. Las asociaciones incorporan cardinalidades explícitas (0..100, 0..10, 0..3) para acotar los topes operativos y garantizar la integridad de las colecciones dinámicas y mapas de búsqueda.

3.3 Modelado Dinámico: Diagramas de Secuencia del Sistema

Para documentar el comportamiento temporal, diseñamos diagramas de secuencia que representan el flujo de ejecución de los casos de uso del sistema. Cada diagrama modela la interacción entre el actor usuario, el objeto controlador (Main), el gestor central (Sistema) y los objetos de dominio específicos. El proceso inicia con la captura de entradas a través de métodos de lectura y la posterior búsqueda o instanciación de objetos.

Mediante el uso de marcos de control condicional, los diagramas representan la validación rigurosa de reglas de negocio: el rechazo automático por duplicidad de credenciales, la verificación de cupos máximos en colecciones, el control de exclusividad de consultorios activos y el cumplimiento estricto de la compatibilidad por especialidad clínica. Asimismo, se ilustra la invocación de métodos sobrescritos que activan el despacho dinámico de operaciones sobre las referencias de las subclases.

3.4 Metodología de Modelado y Verificación

La metodología consistió en la traducción formal de las especificaciones algorítmicas hacia notaciones gráficas normalizadas en UML 2.5. Verificamos que las relaciones de generalización cumplieran con el principio de sustitución y que las multiplicidades reflejasen con fidelidad el comportamiento de las listas dinámicas y tablas de dispersión.

4. Resultados

A continuación, se presentan las evidencias gráficas de los diagramas UML desarrollados para el sistema, organizados por su tipo y contexto arquitectónico:

4.1 Diagramas Estructurales de Clases

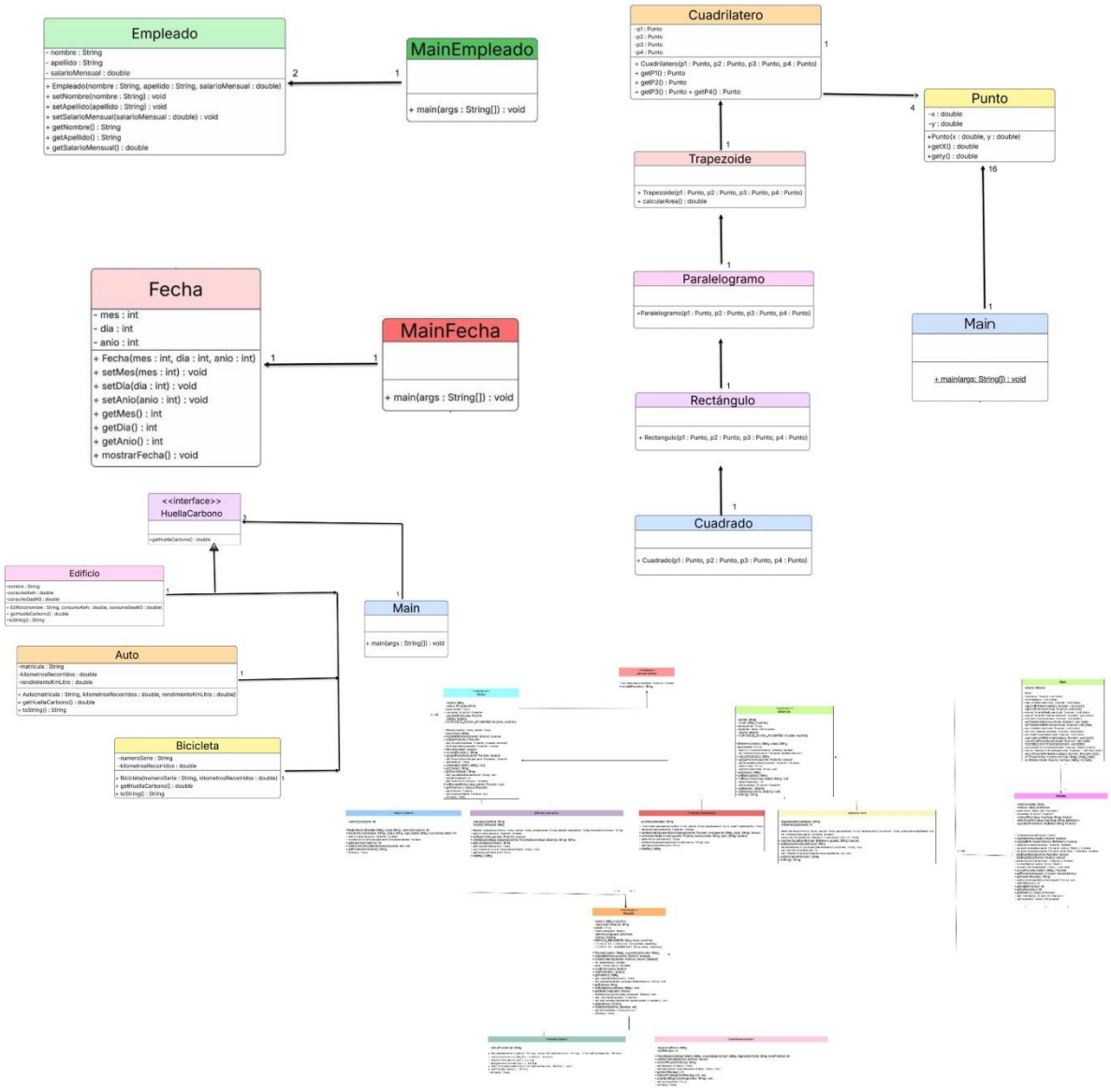


Figura 1: Diagramas estructurales de clases del sistema.

4.2 Diagramas Dinámicos de Secuencia

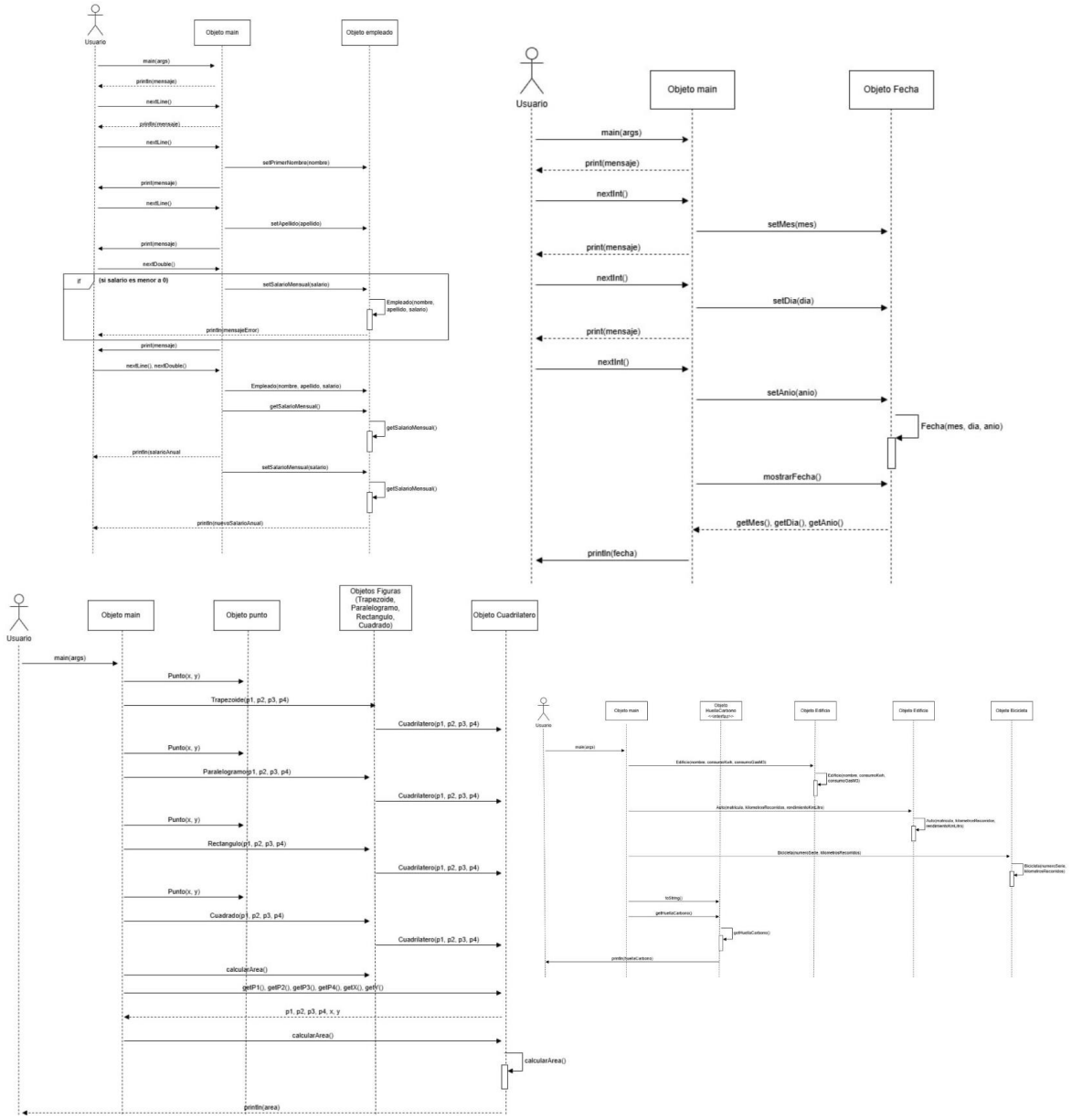


Figura 2: Diagramas de secuencia de las prácticas y módulos iniciales (1 de 4).

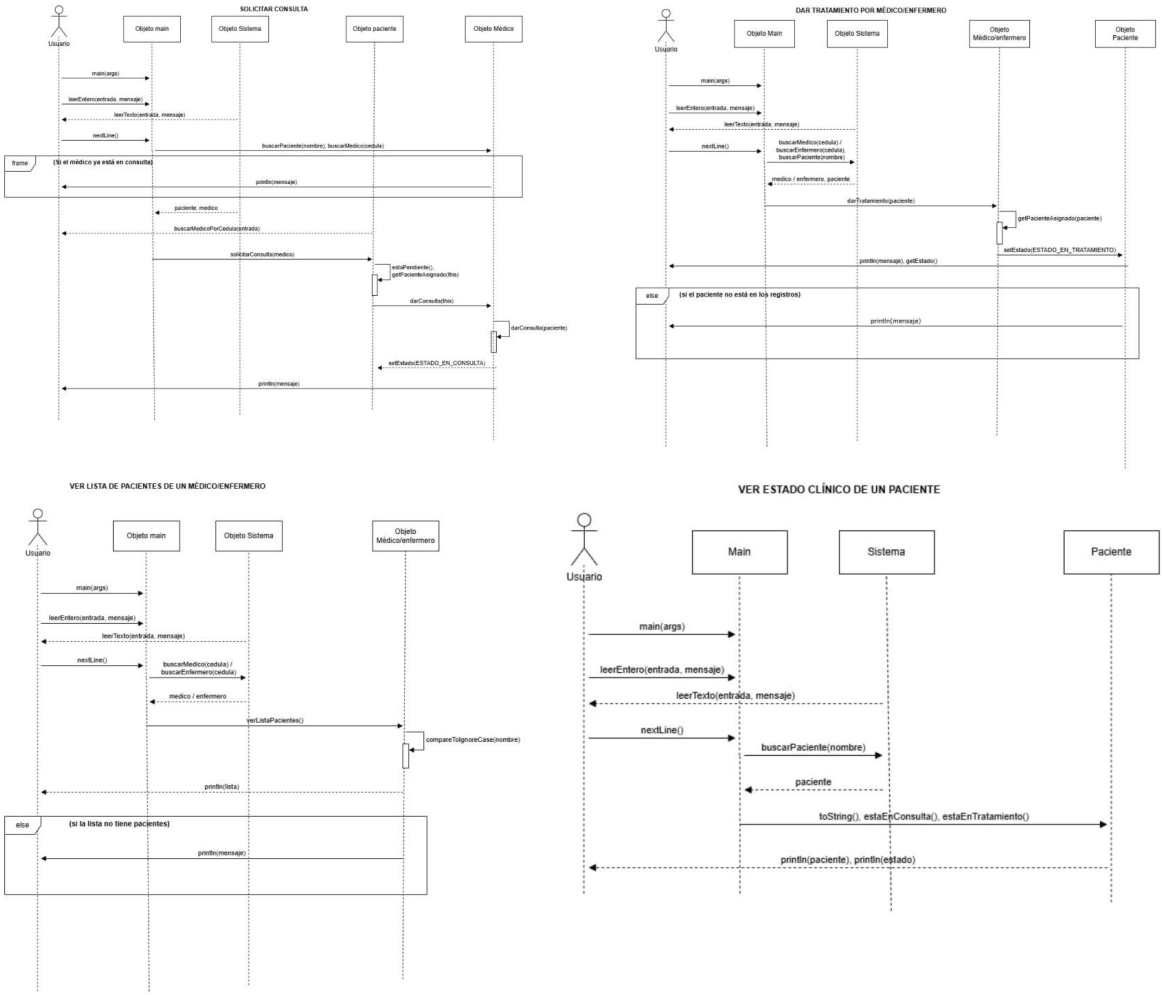


Figura 3: Diagramas de secuencia del sistema hospitalario (2 de 4).

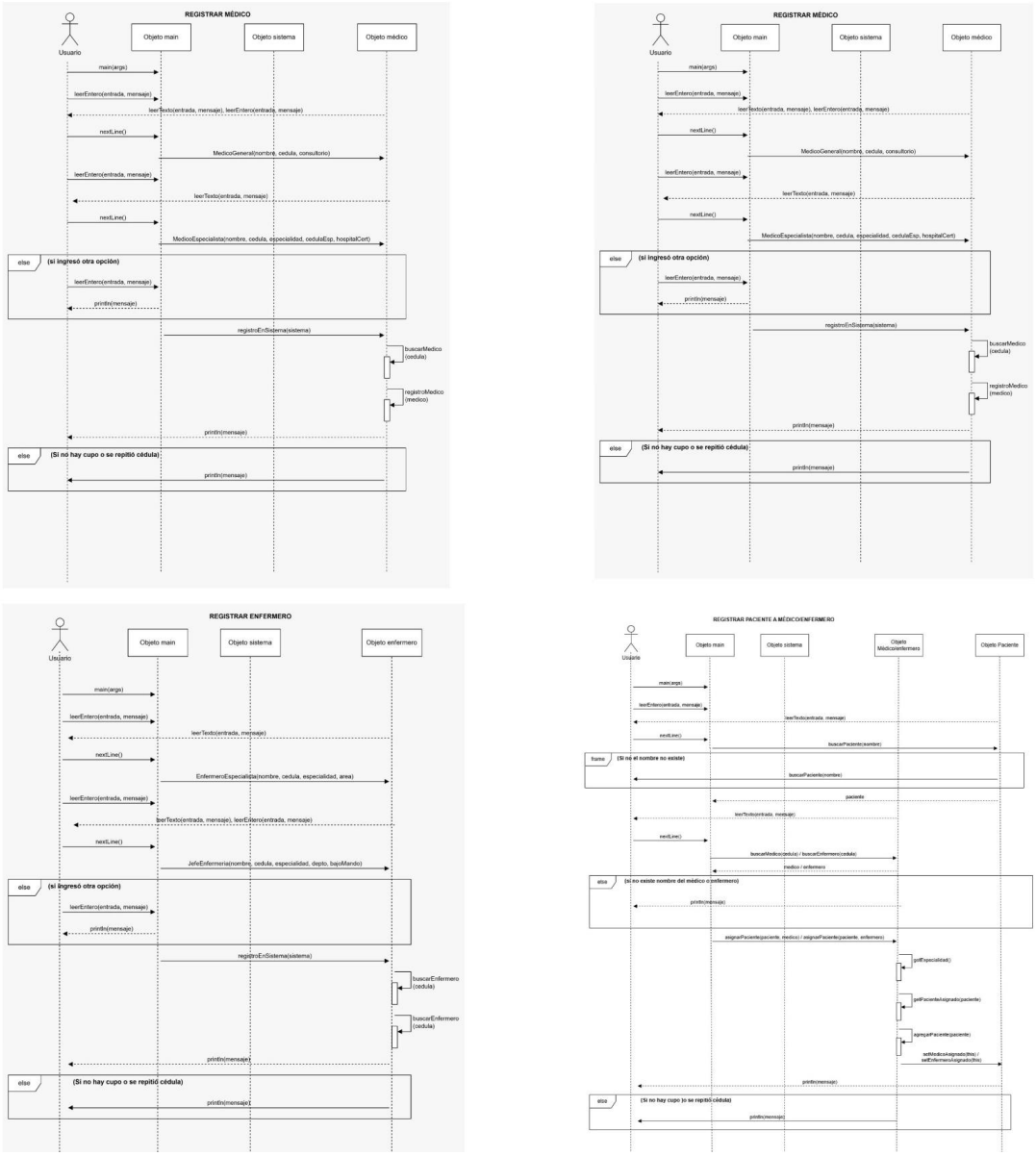


Figura 4: Diagramas de secuencia del sistema hospitalario (3 de 4).

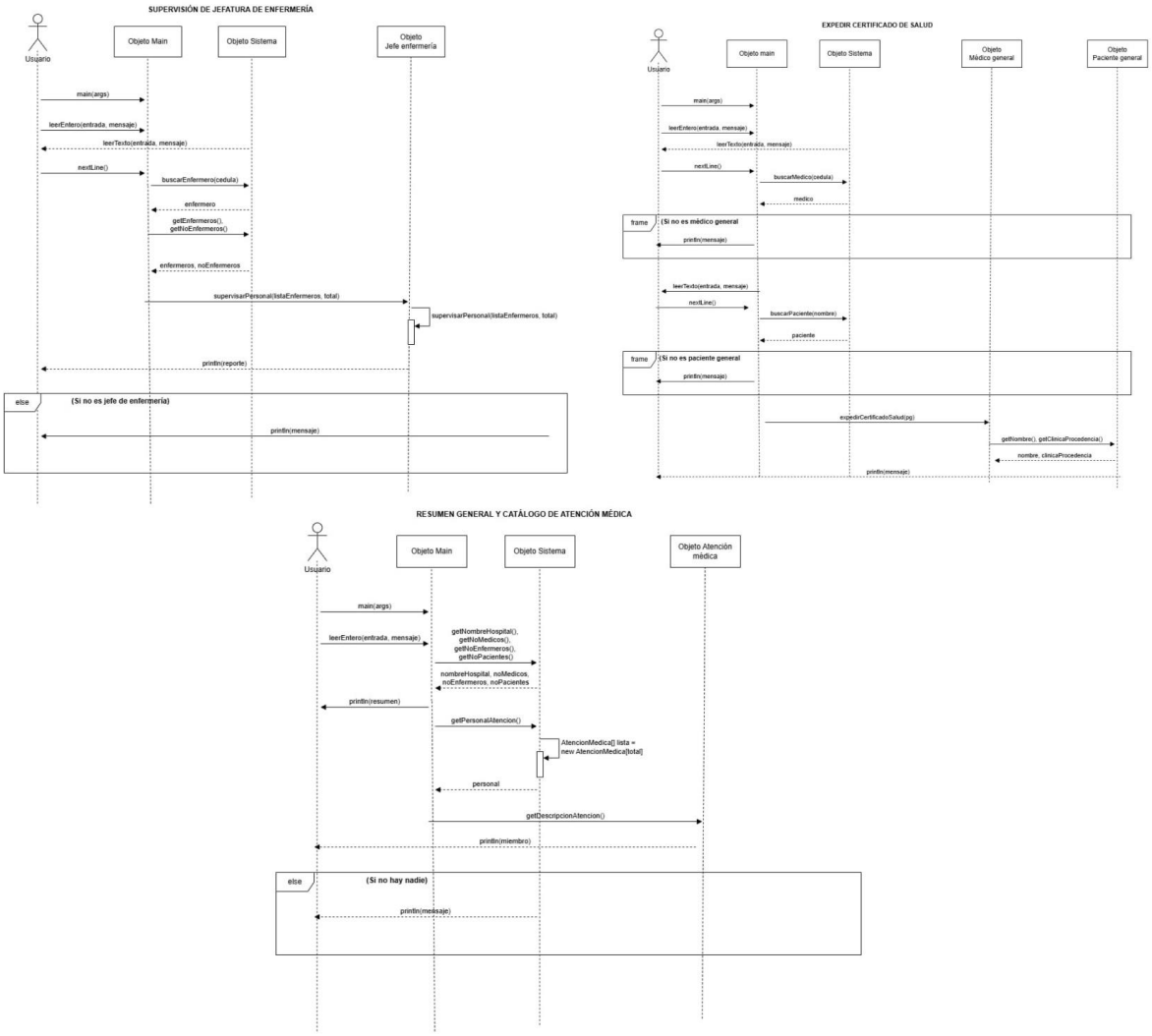


Figura 5: Diagramas de secuencia del sistema hospitalario (4 de 4).

5. Conclusiones

El diseño formal mediante el Lenguaje de Modelado Unificado constituye una herramienta fundamental en la ingeniería de software orientada a objetos, ya que permite estructurar y visualizar las soluciones algorítmicas antes de su codificación. La aplicación rigurosa de los diagramas de clases facilita la correcta distribución de responsabilidades, el establecimiento de jerarquías de herencia y la delimitación estricta de las cardinalidades de asociación, previniendo fallas estructurales en la gestión de memoria y colecciones. De manera complementaria, el modelado dinámico a través de diagramas de secuencia detalla con precisión la cronología de los mensajes, el paso de parámetros y la evaluación de condiciones lógicas en tiempo de ejecución. En conjunto, el uso sistemático de UML estandariza la documentación técnica, asegurando la escalabilidad, el mantenimiento

y la validación formal de los sistemas desarrollados.

6. Referencias

Referencias

[1] Object Management Group (OMG), “Unified Modeling Language (OMG UML), Version 2.5.1”, *OMG Specifications*. [En línea]. Disponible en: https://www.omg.org/spec/UML/2.5.1/

[2] J. A. Solano, “UML”, *Unidades de Apoyo para el Aprendizaje*, CUAED, Facultad de Ingeniería, UNAM, 2020. [En línea]. Disponible en: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3072/mod_resource/content/1/UAPA-UML/index.html

[3] Visual Paradigm, “What is Unified Modeling Language (UML)?”, *Visual Paradigm Visual Modeling Guides*. [En línea]. Disponible en: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/